Data Cleaning & Transformation: NULL Values

1. Identifying NULL Values

Identifying and fixing errors, inconsistencies, and missing values in a dataset.

A. Check for NULL Values

```sql
SELECT * FROM EMP_CLEAN
WHERE Emp_Name IS NULL OR Salary IS NULL OR City IS NULL;
```

B. Check for NOT NULL Values

```sql
SELECT * FROM EMP_CLEAN
WHERE Emp_Name IS NOT NULL AND Salary IS NOT NULL AND City IS NOT NULL;
```

C. Track Row-by-Row Missing Counts

```sql
ALTER TABLE EMP_CLEAN ADD COLUMN null_count INT;

SET SQL_SAFE_UPDATES = 0;
UPDATE EMP_CLEAN
SET null_count = (
    CASE WHEN Emp_Name IS NULL THEN 1 ELSE 0 END +
    CASE WHEN Salary IS NULL THEN 1 ELSE 0 END +
    CASE WHEN City IS NULL THEN 1 ELSE 0 END
);
SET SQL_SAFE_UPDATES = 1;
```

D. Aggregate Column-by-Column

```sql
SELECT
    COUNT(CASE WHEN Emp_ID IS NULL THEN 1 END) AS Emp_ID_Nulls,
    COUNT(CASE WHEN Emp_Name IS NULL THEN 1 END) AS Emp_Name_Nulls,
    COUNT(CASE WHEN Salary IS NULL THEN 1 END) AS Salary_Nulls,
    COUNT(CASE WHEN City IS NULL THEN 1 END) AS City_Nulls
FROM EMP_CLEAN;
```

2. Handling NULL Values (Displaying Defaults)

Use these runtime functions to show fallback values without altering the baseline tables.

Method A: COALESCE (ANSI SQL Standard)

```
SELECT
    Emp_ID,
    COALESCE(Emp_Name, 'Unknown') AS Emp_Name, -- Categorical
    COALESCE(Salary, 0) AS Salary,             -- Numerical
    COALESCE(City, 'Not Specified') AS City
FROM EMP_CLEAN;
```

Method B: IFNULL (MySQL Specific)

```
SELECT
    Emp_ID,
    IFNULL(Emp_Name, 'Unknown') AS Emp_Name,
    IFNULL(Salary, 0) AS Salary,
    IFNULL(City, 'Not Available') AS City
FROM EMP_CLEAN;
```

3. Critical Scenarios: When Not to Fill NULLs

Warning: Replacing NULL values with arbitrary defaults blindly can corrupt data integrity or insert severe analytical bias.

1. When Absence of Data Carries Real Meaning

A NULL value often reflects an expected, active state rather than a failure to collect data.

- *Examples:* An `End_Date` in a live project workflow or a `Termination_Date` for active employees. Forcing dummy dates falsely implies closures.

2. To Avoid Distorting Mathematical Metrics

Filling targets changes denominators and calculation bounds:

- **Averages (AVG):** Aggregate operations drop NULL metrics entirely. Converting NULL salaries to `0` artificially deflates corporate compensation stats.
- **Minimums (MIN):** Mock `0` replacements hijack calculation bounds, rendering true historical floors hidden.

3. Genuinely Unknown Baseline Contexts

If operational entries like a patient's `Blood_Pressure` or credit histories are absent, fabricating fallbacks introduces liability. Leaving fields clear signals data gaps.

4. For Predictive Machine Learning Pipelines

Modern classifiers natively parse missing items via explicit pattern variations (e.g., MNAR - Missing Not At Random). Imputing entries limits algorithmic features.